

## CYBERSECURITY

## LAB 14

## 1. Introduction

This is a continuation of the previous laboratory, this time the main topic will be errors related to the injection of malicious code or content to web requests. There are several types of attacks:

- I. **SQL Injection:** it is a particularly common and dangerous form of injection. The attacker must find the parameter that the application transfers to the database. By typing malicious SQL commands into the content of the parameter, the attacker may trick a web application to forward the query to the database. The consequences are particularly harmful because the attacker may obtain, damage or destroy the contents of the database. An example of such an attack could be an entry into an unsecured login form as email: ' or 1=1- - and anything as a password.

One way to prevent SQL Injection is to use so-called Parameterized Queries (Prepared Statements). Parameterized queries force the programmer to first define the entire SQL code and then pass each parameter to the query. This coding style allows the database to distinguish between code and data, regardless of what the user entered. Another option to protect from SQL Injection is Stored Procedures. It is a very similar option to parameterized queries. The difference between them is that the SQL code for a stored procedure is defined and stored in the database itself and then called from the application. Both these techniques have the same effectiveness in preventing SQL injection.

- II. **NoSQL Injection:** another group of attacks belonging to the wide range of injection type attacks is No-SQL Injection, i.e. attacks on unrelative databases. Non-relational databases such as MongoDB used in the Juice Shop application allow for a slightly looser database structure than typical SQL databases, and are often selected because of their ability to work faster with huge volumes of data and the ease of making changes to database structures. MongoDB is currently one of the most popular non-relational databases, storing data in the form of documents using syntax similar to JSON (JavaScript Object Notation). NoSQL databases are not based on traditional SQL syntax, yet the input values translated into the database queries are "open gate" for attackers, unprotected and unfiltered in no way. The hacker can use the properly thought-out input data to punch, change or delete data, but it can also use any of the functions that operate on the selected NoSQL system, for example to launch a Denial of Service attack, which we will present in this section.

In order to protect against this attack, you should first of all remember to validate the received data, sanitize it or properly filter it, avoid building queries from string objects and also carefully manage the permissions granted to individual user roles.

- III. **Cross-Site Scripting (XSS):** this is an attempt to place code on websites that changes functionality and content for other users. In most cases, such action is aimed at exploiting the vulnerability of the attacker and deviates from the original intentions of the site or application developers. Such attacks can have many different results, from deleting data from an application to preventing the use of the site to steal passwords. The XSS attacks are mainly directed against sites where the user has the ability to enter his own input data - currently most sites have such solutions,

## LAB 14

e.g. searching for goods, adding comments, etc. An XSS attack starts with the injection of a code such as JavaScript into one of the pages that will be visited by the victim. Such actions will trigger a malicious code on the device. In order to achieve this, the attacker adds a code that will be called during the visit.

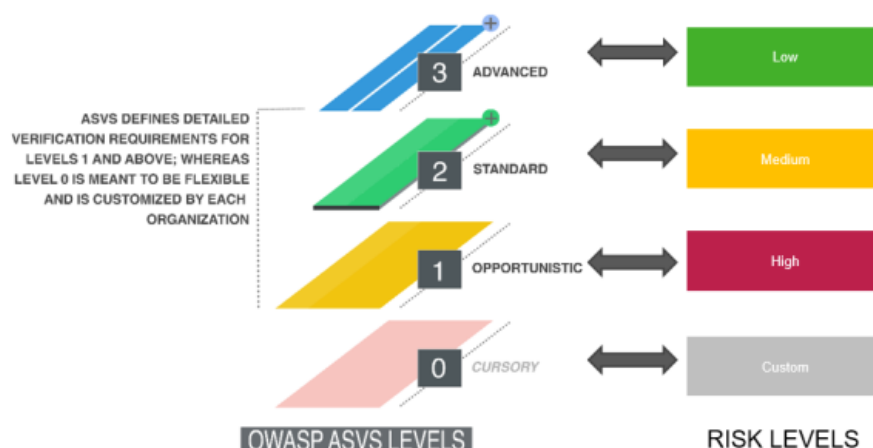
The defense against XSS attacks usually consists of appropriate encoding of data sent by the user, use of sanitization (also recursive, which is especially important when nesting code elements) and protection by validating data on both client and server sides.

- IV. **XML External Entities (XXE):** many older or badly configured XML processors evaluate references to external entities in XML documents. External entities can be used to disclose internal files using the file URI handling procedure, internal file shares, internal port scanning, remote code execution and denial of service attacks.
- V. **Cross-site request forgery (CSRF):** It is an attack that uses the fact of user authentication and authorization in the attack system and uses his privileges to perform unwanted actions on his behalf. It is an attack that uses the fact of user authentication and authorization in the attack system and uses his powers to carry out undesired actions on his behalf. This vulnerability occurs most often when the system uses cookies to recognize whether the user can perform a given action. These cookies are attached to each request sent, regardless of the origin. This allows an attacker to prepare a malicious website that uses the API knowledge of the attacked system to induce the user to perform certain actions (e.g. open photos), but in fact sends requests using his permissions during a session in the attacked system.

All of the listed vulnerabilities are present in the **OWASP TOP 10** and in **Application Security Verification Standard (OWASP ASVS)**.

**OWASP TOP 10** is a widely accepted document presenting the most common and harmful computer security threats. The purpose of the document is to promote awareness and to pay attention to the threats that are most frequently encountered and used to carry out attacks.

**OWASP ASVS** is a list of requirements for application security or testing that can be used by architects, programmers, testers and security engineers. ASVS defines three levels of security verification containing requirements for specific system components. Each of these requirements can be mapped to specific security features and properties that should be built into the software and tested. ASVS is one of the most frequently used security testing methodologies for web applications.



## 2. Required virtual machines

- Kali

## 3. Prerequisites

Get familiar with the following elements:

- OWASP-ZAP
- REST
- Formulas that will be necessary during our labs:
  - ' or 'x'='x
  - ' or 1=1 union select null -
  - <script>alert('XSS')</script>

## 4. Problems and questions

- I. What are the differences between DOM-based, Persistent and Reflected XSS attacks? When is the user exposed to selected types of attacks?
- II. What are typical applications of SQL Injection attacks (what actions can an attacker perform)?
- III. What file types can be used to inject malicious code? Does deserialization of binary files carry the risk of executing malicious code?

## 5. Tasks

- I. Open JuiceShop main page and check if it presents in ZAP history

### SQL Injection

- II. Try to log in using login: ***your\_user\_login' OR '1'='1*** and without password. Click right mouse button on the site, select Inspect Element and find login button. Change property disabled to ***false*** and delete css class ***at-button-disabled*** from the element
- III. Try to find other payloads (maybe easier?) to break authentication mechanism on login page
- IV. Try to find any product with search field in JuiceShop and check the request in ZAP (***juiceshop.domain/rest/products/search?q=***)
- V. Using Request Editor or browser open ***juiceshop.domain/rest/products/search?q=')***--
- VI. Try to get more data (maybe about database schema) with this Blind SQL Injection

### NoSQL Injection

- VII. Log in as any user and add review to any product
- VIII. Find added review and change it
- IX. Find review change request in ZAP (PATCH ***juiceshop.domain/rest/products/reviews***)
- X. Change request body to { "id": { "\$ne": -1 }, "message": "NoSQL Injection!" }

### XSS

- XI. Paste in Search field  
***<iframe src="javascript:alert(`xss`)">***

## LAB 14

- XII. Log in to any user account and provide feedback to administrators (<http://localhost:3000/#/contact> or Customer Feedback option)
- XIII. In the comment box, type:  
`<<script>Foo</script>iframe src="javascript:alert(`xss`)">`
- XIV. Log in as administrator and check [juiceshop.domain/#/administration](http://juiceshop.domain/#/administration)

### XXE

- XV. Prepare XML file with following content:  
`<?xml version="1.0" encoding="ISO-8859-1"?>  
<!DOCTYPE foo [  
<!ELEMENT foo ANY >  
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>  
<foo>&xxe;</foo>`
- XVI. Open **Complaint** subpage
- XVII. Try to upload any .img file, you should get an information that only supported file for upload are PDF and ZIP
- XVIII. In the browser DevTools (F12) open **main-es2015.js** file and find configuration for upload (search for **./file-upload**).
- XIX. Note that in addition to the formats listed in the frontend message, the application also accepts XML files
- XX. Open Console tab in DevTools (it is just to observe the results)
- XXI. Select prepared XML file and upload it (you have to select **All Files** in right bottom part in File Upload window)
- XXII. The console tab should contain an error concerning mimeType. Below should be an answer **410 (Gone)** for POST request

### CSRF

- I. Close ZAP and change proxy settings to **No Proxy**
- II. Ensure that you are logged in as any user
- III. Open new tab in browser and go to <http://htmledit.squarefree.com>
- IV. Paste following code to the upper window  
`<form action="http://localhost:3000/profile" method="POST">  
 <input name="username" value="CSRF"/>  
 <input type="submit"/>  
</form>  
<script>document.forms[0].submit();</script>`
- V. Check if username got changed to "CSRF"